The example in figure 8.6 is a good example of code with poor maintainability:

- The system appears to be so complex it takes a very long time to triage. In my experience if a problem takes longer than an hour to triage, your system is way too complex.
- Development time of fixes is unpredictable. Release 2.3.2 took a much longer time to fix than the other releases; it would be worth drilling in to find out the cause.
- Test cycles that rely heavily on manual testing or a focus on complete regression for small changes are a symptom of a lack of understanding of production behavior.

In figure 8.6 it's hard to tell where to start. If you want to get fixes out faster, it looks like you should:

- Try to improve the test cycle.
- Figure out why it takes so long in triage to understand what's wrong when the system breaks.

Figure 8.7 shows a very different scenario that depicts predictability through a breakdown of MTTR.

Figure 8.7. Another example breaking down MTTR over four releases: with testing fully automated, MTTR goes way down. The next focus for improvement should be the build process.